androsec

EXPLOITATION REPORT

Possible card number

owasp.mstg.uncrackable3

Package owasp.mstg.uncrackable3

Severity High

Generated 2026-11-12 17:51 UTC

Disclaimer

This report is an automated assessment and provides an approximate security overview. It is not guaranteed to be 100% accurate. Some vulnerabilities may not have been detected.

Assessment target

Application and device used for this verification.

App name	owasp.mstg.uncrackable3
Package	owasp.mstg.uncrackable3
Version	1.0
Min / Target SDK	19 / 28
APK size	1.4 MB
SHA-256	6827f776b7d2844342711be6f13695ae8d1a5f209fc0072dbaf8b85e2fcff4b7
Device	VirtualBox
Android	9
Serial	10.0.2.5:5555
Report ID	exploitation_20261112_174946
Generated	2026-11-12 17:51 UTC

Executive summary

Finding severity **High** — app risk **HIGH** — score **40/100**

0	2	6	0	15
Critical	High	Medium	Low	Info

High Possible card number

RULE	LOG_CREDIT_CARD
CONFIDENCE	Medium
MASVS	MASVS-STORAGE-2
EVIDENCE	`F/DEBUG ( 2555): rax 0000000000000000 rbx 00007d07fcdda280 rcx 00007d089f07f827 rdx 0000000000000006`
RECOMMENDATION	Never log PANs.

Assessment write-up

Narrative produced for this finding, including scope, methodology, evidence, and remediation.

VAPT Report: OWASP MSTG UnCrackable3 — LOG_CREDIT_CARD

Executive Summary

The high-severity finding evaluated in this report is **LOG_CREDIT_CARD — Possible card number**.

Static analysis identified a possible Primary Account Number (PAN) in an Android native debug/tombstone log line from `F/DEBUG`. The log line contains numeric/register values that the scanner classified as a possible card number. Dynamic verification was attempted, but the logcat extraction commands failed due to unsupported command options, so the suspected card number was **not live-confirmed**.

If a real PAN is present in debug/crash logs, the business impact is serious: an attacker with ADB access or a process with sufficient log-reading capability could recover cardholder data. This can enable payment card fraud, PCI DSS non-compliance, regulatory exposure, and reputational damage.

The overall application risk was scored **HIGH (40/100)**, and this finding contributes to that risk.

Scope & Authorization

- **Target app:** owasp.mstg.uncrackable3
- **Package:** owasp.mstg.uncrackable3
- **Version:** 1.0
- **Target SDK:** 28
- **APK size:** 1.4 MB
- **SHA-256:** `6827f776b7d2844342711be6f13695ae8d1a5f209fc0072dbaf8b85e2fcff4b7`
- **Overall app risk:** HIGH — 40/100

`{'Critical': 0, 'High': 2, 'Medium': 6, 'Low': 0, 'Info': 15}`

Authorization note:
This assessment was performed as part of a university graduation project against the intentionally vulnerable OWASP MSTG UnCrackable3 application. Testing was conducted on a lab-owned Android emulator. No production systems, production data, or third-party systems were involved.

OWASP Mobile Top 10 Classification

OWASP Mobile Top 10: M2 - Insecure Data Storage

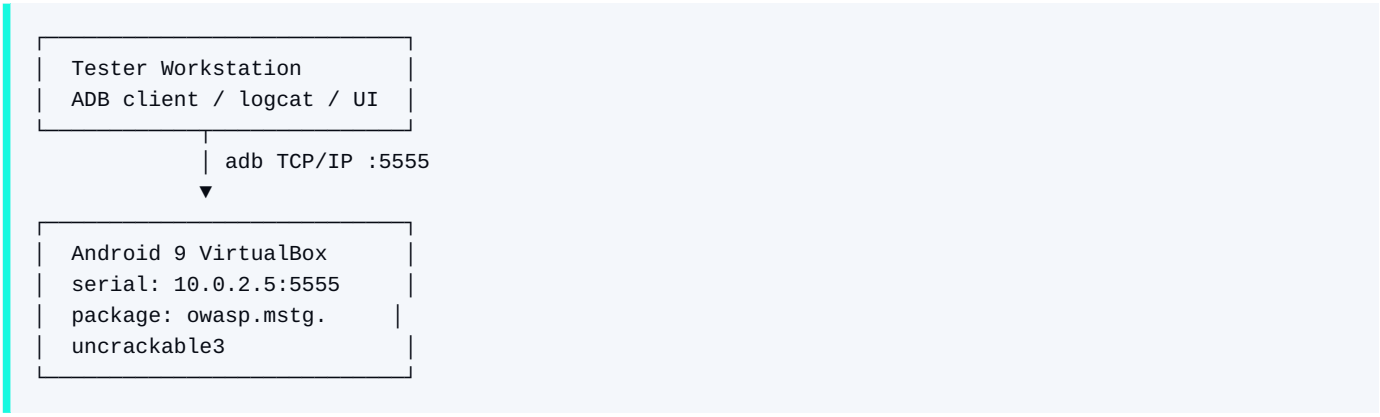
This finding is classified as **M2 - Insecure Data Storage** because it involves sensitive data being written to Android system logs. The `F/DEBUG` tombstone/log stream is outside the application container and may be readable through ADB or by other privileged processes. Writing a possible card number to such logs is an insecure handling of sensitive data and aligns with:

- **MASVS-STORAGE-2:** “No sensitive data is written to application logs.”

The scanner rule `LOG_CREDIT_CARD` specifically flags suspected PAN-type numeric sequences in log output, which falls under this mobile risk category.

Environment & Topology

- **Tester host:** Graduation project analysis workstation running ADB client
- **ADB transport:** TCP/IP to `10.0.2.5:5555`
- **Device:** VirtualBox Android 9
- **Device serial:** `10.0.2.5:5555`



Methodology

The assessment followed this flow:

1. Static analysis was used to identify the `LOG_CREDIT_CARD` finding and produce the quoted static evidence.
2. Dynamic verification was attempted by launching the target app, waiting for the native crash/debug path, and extracting logcat/tombstone output.
3. Evidence capture was performed through screenshots and command output.

Due to tooling errors during logcat extraction, dynamic confirmation of the possible card number was not completed.

Finding Detail

FIELD	VALUE
Rule ID	`LOG_CREDIT_CARD`
Severity	High
Confidence	Medium
MASVS	MASVS-STORAGE-2
Static evidence	`F/DEBUG ( 2555): rax 0000000000000000 rbx 00007d07fcdda280 rcx 00007d089f07f827 rdx 0000000000000006`
Static recommendation	Never log PANs.

The above line is quoted verbatim as the static-analysis evidence. The scanner flagged it as containing a possible card number. The quoted line contains hexadecimal-style register values from an `F/DEBUG` native debug tombstone rather than a plaintext 13–19 digit PAN, so dynamic verification was required.

Exploitation / Verification Steps

The following steps were executed:

1. **Step 1 — Capture baseline UI before starting the target app**
   Command: `screenshot`
   Result: `Screenshot saved as screenshot_01.png`
2. **Step 2 — Force-stop the target app to ensure a clean launch**
   Command: `adb shell am force-stop owasp.mstg.uncrackable3`
   Result: `(exit 0, no output)`
3. **Step 3 — Clear logcat buffers to isolate new logs**
   Command: `adb logcat -c`
   Result: `failed to clear the 'crash' log`
4. **Step 4 — Launch the target app to trigger the native crash/root-detection path**
   Command: `adb shell monkey -p owasp.mstg.uncrackable3 1`
   Result:
   `bash arg: -p bash arg: owasp.mstg.uncrackable3 bash arg: 1 Events injected: 1 ## Network stats: elapsed time=15ms (0ms mobile, 0ms wifi, 15ms not connected)`
5. **Step 5 — Wait for the crash and debuggerd tombstone to be written**
   Command: `adb shell sleep 5`
   Result: `(exit 0, no output)`
6. **Step 6 — Extract the F/DEBUG tombstone containing register values flagged as possible card number**
   Command: `adb logcat -d -v long | grep -A 30 -B 5 'F/DEBUG'`
   Result: Failed. The command produced `Unrecognized Option A` and printed logcat usage. No `F/DEBUG` lines were extracted.
7. **Step 7 — Extract any card-like digit sequence from the captured tombstone/logcat**
   Command: `adb logcat -d -v long | grep -oE '[0-9]{13,19}' | tail -n 20`
   Result: Failed. The command produced `Unrecognized Option o` and printed logcat usage. No card-like digit sequence was extracted.
8. **Step 8 — Capture result/proof screenshot showing the crashed app or crash dialog**
   Command: `screenshot`
   Result: `Screenshot saved as screenshot_08.png`

Evidence

The following evidence artifacts were collected:

- **screenshot_01.png** — Step 1 baseline UI before launching the target app.
- **Command output from Step 4** — Evidence of app launch via `adb shell monkey -p owasp.mstg.uncrackable3 1`.
- **screenshot_08.png** — Step 8 result screenshot showing the crashed app or crash dialog.

The following steps produced no usable evidence due to command failure:

- Step 6 — `F/DEBUG` tombstone extraction failed.
- Step 7 — Card-like digit sequence extraction failed.

Remediation

The following developer-actionable fixes are recommended:

1. **Stop logging PAN/sensitive card data.** Remove any logging statements that write full card numbers, PAN-like data, or sensitive register values to logcat, crash logs, or debug output.
2. **Sanitize native crash logging.** Ensure native crash handlers and debuggerd/tombstone output do not dump raw numeric data that may contain sensitive payment information.
3. **Use centralized log redaction.** Implement a logging utility that automatically masks or redacts PAN-like patterns before output reaches logcat or files.
4. **Use tokenization or masking.** If payment cards must be referenced, store or log only a token or a masked value such as `**** **** **** 1234`, never the full PAN.
5. **Remove verbose debug logging from release builds.** Ensure `F/DEBUG` /native debug logging is disabled or stripped in production builds.
6. **Add regression checks.** Add tests that fail if a PAN-like sequence appears in captured logcat/ tombstone output.

Conclusion

Static analysis identified a high-severity possible card number exposure in `F/DEBUG` log output. Dynamic verification was not completed because the logcat extraction commands failed with unsupported options. Therefore, residual risk remains **HIGH** until the issue is verified and remediated.

Follow-up actions:

- Re-run dynamic log capture using supported syntax, for example:
  `adb logcat -d -v long > logcat.txt`
  then inspect `logcat.txt` locally with grep.
- Determine whether the flagged `F/DEBUG` line contains an actual PAN or is a false positive from hexadecimal register values.
- If a real PAN is confirmed, apply the remediation steps above and retest.
- Repeat the verification in a clean lab environment with correctly isolated logcat buffers.

Executed steps

Commands that ran and the output captured for each one.

1. Capture baseline UI before starting the target app

COMMAND

```
screenshot
```

RESULT

```
Screenshot saved as screenshot_01.png
```

2. Force-stop the target app to ensure a clean launch

COMMAND

```
adb shell am force-stop owasp.mstg.uncrackable3
```

RESULT

```
(exit 0, no output)
```

3. Clear logcat buffers to isolate new logs

COMMAND

```
adb logcat -c
```

RESULT

```
failed to clear the 'crash' log
```

4. Launch the target app to trigger the native crash/root-detection path

COMMAND

```
adb shell monkey -p owasp.mstg.uncrackable3 1
```

RESULT

```
bash arg: -p
  bash arg: owasp.mstg.uncrackable3
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=15ms (0ms mobile, 0ms wifi, 15ms not connected)
```

5. Wait for the crash and debuggerd tombstone to be written

COMMAND

```
adb shell sleep 5
```

RESULT

```
(exit 0, no output)
```

6. Extract the F/DEBUG tombstone containing register values flagged as possible card number

COMMAND

```
adb logcat -d -v long | grep -A 30 -B 5 'F/DEBUG'
```

RESULT

```
Unrecognized Option A
Usage: logcat [options] [filterspecs]
options include:
  -s              Set default filter to silent. Equivalent to filterspec '*:S'
  -f <file>, --file=<file>               Log to file. Default is stdout
  -r <kbytes>, --rotate-kbytes=<kbytes>
                  Rotate log every kbytes. Requires -f option
  -n <count>, --rotate-count=<count>
                  Sets max number of rotated logs to <count>, default 4
  --id=<id>       If the signature id for logging to file changes, then clear
                  the fileset and continue
  -v <format>, --format=<format>
                  Sets log print format verb and adverbs, where <format> is:
                    brief help long process raw tag thread threadtime time
                  and individually flagged modifying adverbs can be added:
                    color descriptive epoch monotonic printable uid
                    usec UTC year zone
                  Multiple -v parameters or comma separated list of format and
                  format modifiers are allowed.
  -D, --dividers  Print dividers between each log buffer
  -c, --clear     Clear (flush) the entire log and exit
                  if Log to File specified, clear fileset instead
  -d              Dump the log and then exit (don't block)
  -e <expr>, --regex=<expr>
                  Only print lines where the log message matches <expr>
                  where <expr> is a Perl-compatible regular expression
  -m <count>, --max-count=<count>
```

7. Extract any card-like digit sequence from the captured tombstone/logcat

COMMAND

```
adb logcat -d -v long | grep -oE '[0-9]{13,19}' | tail -n 20
```

RESULT

```
Unrecognized Option o
Usage: logcat [options] [filterspecs]
options include:
```

```
  -s              Set default filter to silent. Equivalent to filterspec '*:S'
  -f <file>, --file=<file>               Log to file. Default is stdout
  -r <kbytes>, --rotate-kbytes=<kbytes>
                  Rotate log every kbytes. Requires -f option
  -n <count>, --rotate-count=<count>
                  Sets max number of rotated logs to <count>, default 4
  --id=<id>       If the signature id for logging to file changes, then clear
                  the fileset and continue
  -v <format>, --format=<format>
                  Sets log print format verb and adverbs, where <format> is:
                    brief help long process raw tag thread threadtime time
                  and individually flagged modifying adverbs can be added:
                    color descriptive epoch monotonic printable uid
                    usec UTC year zone
                  Multiple -v parameters or comma separated list of format and
                  format modifiers are allowed.
  -D, --dividers  Print dividers between each log buffer
  -c, --clear     Clear (flush) the entire log and exit
                  if Log to File specified, clear fileset instead
  -d              Dump the log and then exit (don't block)
  -e <expr>, --regex=<expr>
                  Only print lines where the log message matches <expr>
                  where <expr> is a Perl-compatible regular expression
  -m <count>, --max-count=<count>
```

8. Capture result/proof screenshot showing the crashed app or crash dialog

COMMAND

```
screenshot
```

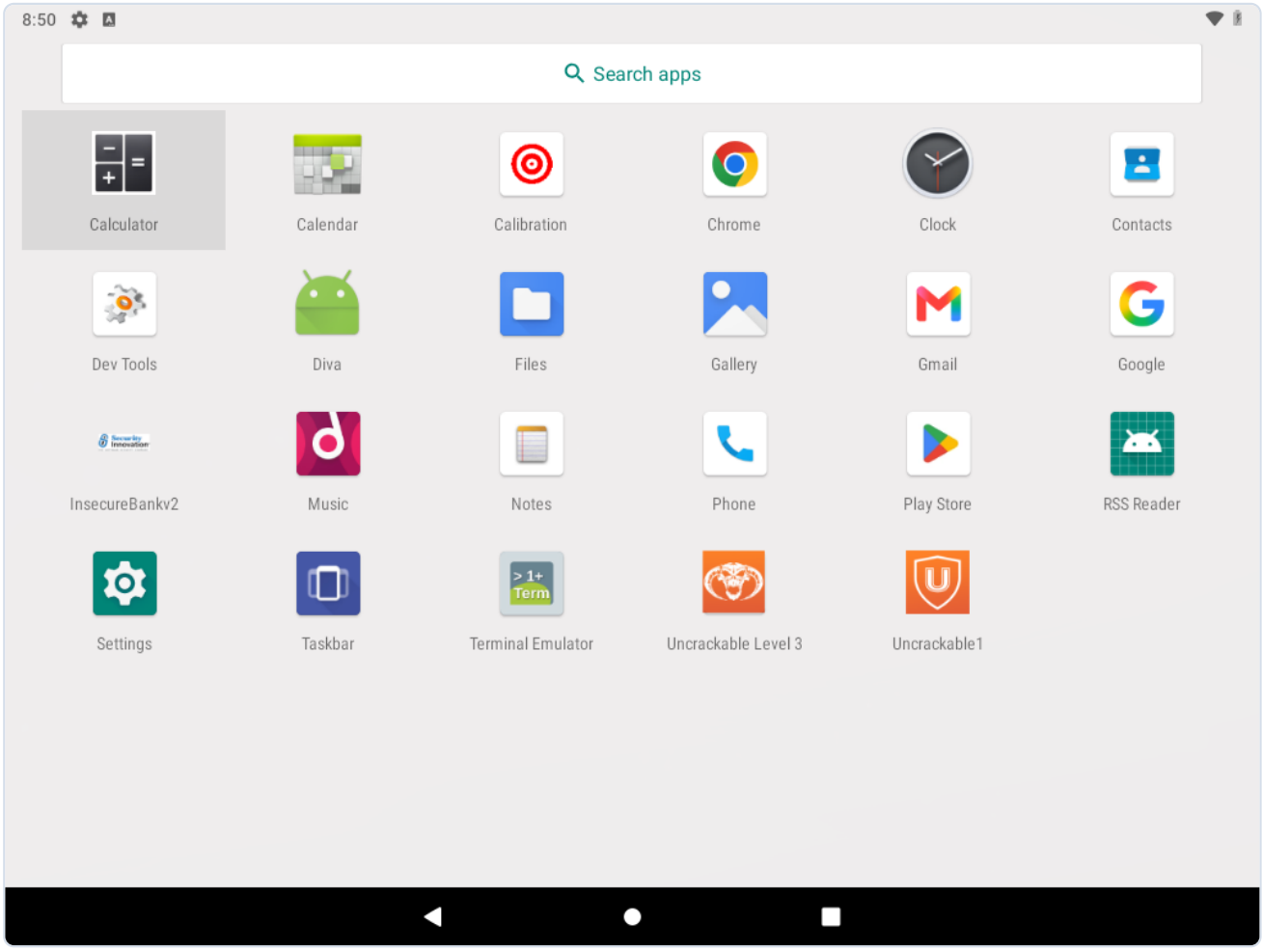
RESULT

```
Screenshot saved as screenshot_08.png
```

Captured evidence

Screenshots and command output saved during the run.

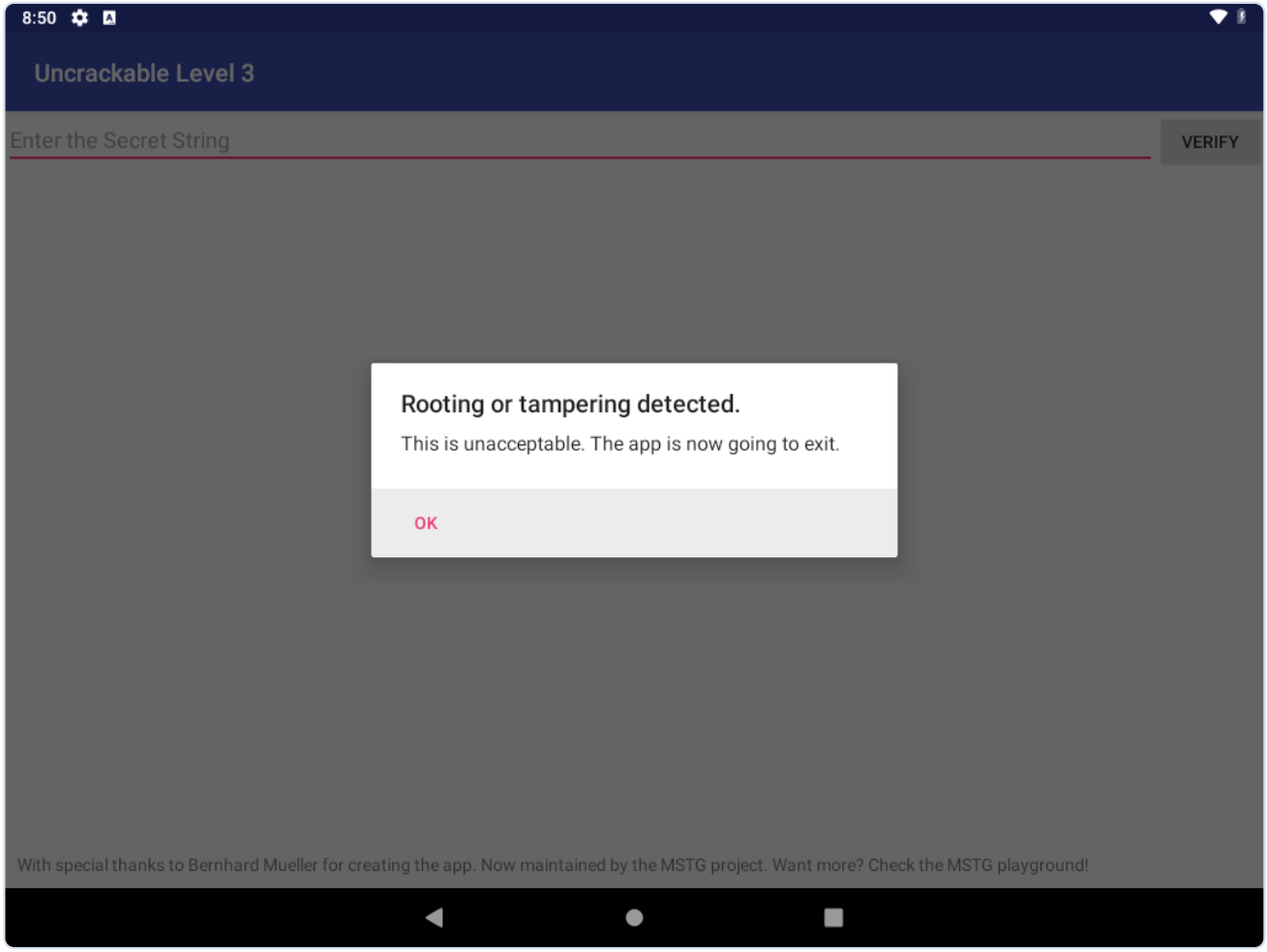
Screenshot — Capture baseline UI before starting the target app



Command output — Launch the target app to trigger the native crash/root-detection path

```
bash arg: -p
  bash arg: owasp.mstg.uncrackable3
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=15ms (0ms mobile, 0ms wifi, 15ms not connected)
```

Screenshot — Capture result/proof screenshot showing the crashed app or crash dialog



DISCLAIMER

This report is an automated assessment and provides an approximate security overview. It is not guaranteed to be 100% accurate. Some vulnerabilities may not have been detected.